جامعة أم القرى
UMM AL-QURA UNIVERSITY

ML project Report:

Game Difficulty Adjuster

Group Name: ML project 1
Section: 2

Group Members:

NAME	ID
Reham Khan	445013974
Batoul Hallaq	445013931
Afrah Ayob	445013880
Yosra Tamer	445013908
Rawaf Tazi	445003909

Dr. Afaf Almehmadi

Table of Contents:

1. Abstract 4
2. Introduction 5
3. Problem Definition 5
4. Literature Review 6
   4.1 Dynamic Difficulty Adjustment via Fast User Adaptation [1] 6
      4.1.1 Problem Addressed 6
      4.1.2 Data Used 6
      4.1.3 Algorithms and Methods Used 6
      4.1.4 Proposed Solution 6
      4.1.5 Results and Evaluation 6
      4.1.6 Strengths and Weaknesses 6

   4.2 Dynamic Difficulty Adjustment Based on Skills Classification [2] 7
      4.2.1 Problem Addressed 7
      4.2.2 Data Used 7
      4.2.3 Algorithms and Methods Used 7
      4.2.4 Proposed Solution 7
      4.2.5 Results and Evaluation 7
      4.2.6 Strengths and Weaknesses 7

   4.3 Dynamic Difficulty Adjustment with UX and Gameplay Constraints [3] 8
      4.3.1 Problem Addressed 8
      4.3.2 Data Used 8
      4.3.3 Algorithms and Methods Used 8
      4.3.4 Proposed Solution 8
      4.3.5 Results and Evaluation 8
      4.3.6 Strengths and Weaknesses 8

   4.4 Dynamic Difficulty Adjustment in a Visual Working Memory Game [4] 9
      4.4.1 Problem Addressed 9
      4.4.2 Data Used 9
      4.4.3 Algorithms and Methods Used 9
      4.4.4 Proposed Solution 9
      4.4.5 Results and Evaluation 9
      4.4.6 Strengths and Weaknesses 9

   4.5 Personalized Game Difficulty Prediction Using Factorization Machines [5] 10
      4.5.1 Problem Addressed 10
      4.5.2 Data Used 10
      4.5.3 Algorithms and Methods Used 10
      4.5.4 Proposed Solution 10
      4.5.5 Results and Evaluation 10
      4.5.6 Strengths and Weaknesses 10

   4.6 Dynamic Difficulty Adjustment of Serious Game Based on Synthetic Fog [6] 11
      4.6.1 Problem Addressed 11
      4.6.2 Data Used 11
      4.6.3 Algorithms and Methods Used 11
      4.6.4 Proposed Solution 11
      4.6.5 Results and Evaluation 11
      4.6.6 Strengths and Weaknesses 11

4.7 Fast Game Content Adaption Through Bayesian-Based Player Modeling [7] 12
4.7.1 Problem Addressed 12
4.7.2 Data Used 12
4.7.3 Algorithms and Methods Used 12
4.7.4 Proposed Solution 12
4.7.5 Results and Evaluation 12
4.7.6 Strengths and Weaknesses 12

4.8 Dynamic Difficulty Adjustment in Platform Games [8] 13
4.8.1 Problem Addressed 13
4.8.2 Data Used 13
4.8.3 Algorithms and Methods Used 13
4.8.4 Proposed Solution 13
4.8.5 Results and Evaluation 13
4.8.6 Strengths and Weaknesses 13

4.9 Emotion-Based Dynamic Difficulty Adjustment in Video Games [9] 14
4.9.1 Problem Addressed 14
4.9.2 Data Used 14
4.9.3 Algorithms and Methods Used 14
4.9.4 Proposed Solution 14
4.9.5 Results and Evaluation 14
4.9.6 Strengths and Weaknesses 14

4.10 Engagement-Oriented Dynamic Difficulty Adjustment (EDDA) [10] 15
4.10.1 Problem Addressed 15
4.10.2 Data Used 15
4.10.3 Algorithms and Methods Used 15
4.10.4 Proposed Solution 15
4.10.5 Results and Evaluation 15
4.10.6 Strengths and Weaknesses 16

4.11 Comparative Analysis 17

4.12 Gap Analysis and Justification 18
4.12.1 Justification for the Current Project 19

5. Detailed Machine Learning Module 20
5.1 Creating Synthetic Data 20, 21
5.2 Feature Aggregation and Player Profiling 22
5.3 Player Clustering (Skill Grouping) 23, 24
5.4 Feature Engineering and Target Creation 25, 26
5.5 Supervised Learning Model (Random Forest) 26, 27, 28
5.6 Before and After Simulation (DDA Effect) 29, 30
5.7 Summary of the ML Pipeline 31

6. Conclusion 31

7. References 32, 33

1- Abstract:

Serious puzzle games often struggle to maintain player engagement when difficulty levels remain static and fail to adjust to individual performance. This project addresses this challenge by proposing a machine-learning driven adaptive difficulty framework that personalizes the gameplay experience for puzzle-game players. A synthetically generated yet statistically realistic dataset of 30,000 gameplay records was created to simulate diverse player behaviors across varying skill levels, incorporating metrics such as accuracy, solving time, hints, attempts, and learning progression. K-Means clustering was applied to identify five distinct player-performance segments, forming the basis for player modeling.

A Random Forest classifier was then trained to predict the optimal difficulty level for each player using both short-term in-game indicators and long-term skill-development patterns. A simulation module was developed to evaluate performance before and after implementing dynamic difficulty adjustment.

Results demonstrate that the proposed framework effectively identifies skill tiers and delivers personalized difficulty recommendations that maintain an optimal challenge level, reduce frustration, and enhance overall engagement.

This work contributes a scalable and data-driven adaptive system that can be integrated into various puzzle-game environments and expanded to support real-time dynamic difficulty updates.

Keywords: adaptive difficulty, puzzle games, player modeling, dynamic difficulty adjustment, clustering, classification.

Code link: [click here]

2- Introduction:

Puzzle games are commonly used for entertainment, education, and cognitive training. However, one of the main challenges in designing these games is keeping the difficulty level suitable for different types of players. When the difficulty is too high, players may feel frustrated and lose motivation to continue. On the other hand, if the game is too easy, players may get bored and stop engaging with the content. Because of this, creating a system that can adjust the difficulty based on each player's behavior has become an important goal in modern game design and interactive technologies.

Adaptive difficulty systems work by observing how the player performs and then updating the challenge level to match their abilities. Machine learning provides effective methods for understanding player patterns, identifying skill levels, and predicting what difficulty each player needs. Although previous research has explored different techniques such as clustering, reinforcement learning, and classification models, many of these approaches face limitations, including small datasets or difficulty adapting to new players.

In this project, we develop a complete machine-learning system that adjusts puzzle-game difficulty. We create a large simulated dataset that reflects realistic player behavior, extract meaningful performance features, group players based on their skill levels, and train a model that can recommend the most appropriate difficulty level. Finally, we test the system by comparing the player's performance before and after applying the adaptive model. The proposed solution aims to offer a more personalized and balanced gameplay experience that better matches each player's abilities.

3- Problem Definition:

Static difficulty systems in puzzle games often fail to accommodate variations in player skill, learning speed, and behavioral patterns. When all players receive the same difficulty level, weaker players may struggle and become frustrated, while stronger players may lose engagement due to insufficient challenge. This mismatch reduces overall performance, lowers satisfaction, and negatively impacts the learning or entertainment value of the game. To address this issue, there is a need for an adaptive difficulty mechanism capable of analyzing player behavior and assigning a personalized challenge level.

The problem this project aims to solve is how to design a machine-learning-based system that predicts the optimal difficulty for each player by using gameplay metrics such as accuracy, solving time, attempts, hints, and long-term performance trends. The goal is to build a model that can correctly identify player skill categories and can be integrated into games to provide real-time difficulty adjustments that improve balance, engagement, and success rates across diverse player types.

4- Literature Review:

4.1 Dynamic Difficulty Adjustment via Fast User Adaptation [1]

4.1.1 Problem Addressed

This study addresses the challenge of automatically predicting player skill and adjusting game difficulty in real-time. Previous game difficulty adjustment systems often struggle to adapt precisely to each player's performance. They may require manual feature design, fail to handle new or sparse data effectively, or produce abrupt changes in difficulty that reduce player enjoyment. This study aims to develop a system that provides smooth, personalized difficulty adjustment for each player to enhance engagement and satisfaction.

4.1.2 Data Used

This study uses gameplay data including player scores, completion times, mistakes, and in-game actions. The data comes from both simulated and real human players to evaluate system performance.

4.1.3 Algorithms and Methods Used

- Fuzzy Logic: maps player performance to difficulty levels.
- Deep Learning: feed-forward neural networks.
- Reinforcement Learning: Proximal Policy Optimization (PPO) for dynamic difficulty adjustment.

4.1.4 Proposed Solution

- Adjusting game difficulty automatically based on predictions.
- Using fuzzy logic for smooth adaptation.
- Applying ML and RL-based methods to learn from player behavior for precise adjustments.

4.1.5 Results and Evaluation

- Adaptive systems improved.
- Difficulty adjustments were smooth without sudden spikes or drops.
- Fuzzy logic and ML/RL methods outperformed simple rule-based systems.

4.1.6 Strength and Weakness

Strengths:

- Personalized difficulty adjustment
- Smooth adaptation
- Works for various game types

Weaknesses:

- Requires carefully designed rules or features
- May not generalize well to very different games

4.2 Dynamic Difficulty Adjustment Based on Skills Classification [2]

4.2.1 Problem Addressed

This study investigates how to enhance player engagement by using deep learning to classify player skill level and automatically adjust game difficulty. Traditional DDA systems often fail to match challenge to skill accurately, leading to boredom or frustration.

4.2.2 Data Used

This study uses a dataset of 32,280 records from 56 players across 148 game sessions, including player performance metrics, interactions with game objects, and game progress.

4.2.3 Algorithms and Methods Used

- Feed-forward neural networks for skill classification.
- A two-step method: first classify player skill, then apply dynamic difficulty adjustment (DDA).
- Comparison with baseline methods lacking personalization.

4.2.4 Proposed Solution

- Real-time classification of player's expertise into categories (high, medium, low).
- Automatic adjustment of game difficulty based on the classified skill level.
- Integration of the deep learning model into the game to deliver a personalized challenge.

4.2.5 Results and Evaluation

- The model achieved high accuracy in skill classification and led to enhanced player immersion.
- Personalized difficulty adjustment outperformed non-personalized baselines in maintaining challenge and user satisfaction.

4.2.6 Strength and Weakness

Strengths:

- High accuracy in classifying player skill.
- Personalized difficulty adaptation improves player experience.
- Demonstrated in a real game setting with a substantial dataset.

Weaknesses:

- Tested on a single game; generalizability to other game types is unclear.
- Difficulty adjustments occurred only after game sessions, not continuously during play.

4.3 Dynamic Difficulty Adjustment with UX and Gameplay constraints [3]

4.3.1 Problem Addressed

The paper tackles the challenge of dynamically adjusting game difficulty (DDA) in a way that optimizes each player's experience while also maintaining balance across all players. Existing methods focus on individuals only and ignore global constraints especially completion rate which is hard to integrate into neural networks because it is non-differentiable.

4.3.2 Data Used

The system uses large-scale behavioral data from about one million players:
• Player actions and game states over a two-week period.
• Actual performance values that reflect the effective difficulty experienced.
• Players are grouped into homogeneous clusters to ensure meaningful comparisons.

4.3.3 Algorithms and Methods Used

• A deep fully connected neural network (five hidden layers) to predict optimal difficulty.
• K-Means clustering to group similar players.
• A modified projected gradient descent method to incorporate completion rate constraints.
•A custom UX loss function balancing personal fit and similarity within each cluster.

4.3.4 Proposed Solution

• A UX loss that encourages both personalized difficulty and consistency among similar players.
• A projection-based method to enforce completion-rate constraints inside the neural network.

4.3.5 Results and Evaluation

• In an A/B test with 1M players:
• Target completion rate: 8–10%
• Rule-based system: 12%
• Their model: 8.7%, matching the target much more closely.
• Their method also significantly reduced variance in completion rates across clusters, showing better global control and stability.

4.3.6 Strengths and Weaknesses

Strengths:
• Generic UX formulation applicable to multiple games.
• Successfully integrates a non-differentiable constraint.
• Highly scalable and reduces manual tuning by designers.
• Strong empirical performance on a large user base.

Weaknesses:
• Difficulty cannot change mid-period; a wrong assignment persists.
• Relies on large datasets and clustering, which may limit applicability to smaller games.

• Clustering may oversimplify differences between individual players.

4.4 Dynamic Difficulty Adjustment in a Visual Working Memory Game [4]

4.4.1 Problem Addressed

The paper addresses how to perform Dynamic Difficulty Adjustment (DDA) for a visual working memory (VWM) game when difficulty is continuous rather than discrete. Traditional RL-based DDA methods cannot handle such continuous difficulty spaces, so the authors aim to build a continuous RL system that adjusts difficulty smoothly and more precisely based on player performance.

4.4.2 Data Used

• Simulated player agents for RL training.
• 52 human participants for real-world evaluation.
• Collected data includes trial scores, selected difficulty values, and Game Experience Questionnaire (GEQ) responses.

4.4.3 Algorithms and Methods Used

• Proximal Policy Optimization (PPO) as the reinforcement learning algorithm.
• A continuous difficulty representation created from three task features.
They compare their approach with two rule-based baselines.

4.4.4 Proposed Solutions

The solution is a continuous RL-driven DDA system that selects the next task's difficulty based on recent player performance. It combines simulated training with human fine-tuning and uses PPO to operate directly in a continuous action space.

4.4.5 Results and Evaluation

Across 52 human players:
• The RL method achieved higher scores and win rates than the rule-based systems.
• GEQ ratings showed better user experience under the RL approach.
• Difficulty increased smoothly without major drops in performance, showing stable adaptation.

4.4.6 Strengths and Weaknesses

Strengths:
• Handles continuous difficulty spaces effectively.
• Improves game performance and subjective experience.
• More personalized and stable than rule-based methods.

Weaknesses:
• Requires a massive difficulty database.
• Difficulty metric is hand-crafted and may not generalize.
• Real-world fine-tuning is still needed

4.5 Personalized Game Difficulty Prediction Using Factorization Machines [5]

4.5.1 Problem Addressed

This paper looks at how to predict how hard a mobile puzzle game level will be for each player. The main idea is to estimate how many tries a player needs to finish a level and understand both player skill and level difficulty, so predictions are better than just using a general average.

4.5.2 Data Used

The data comes from a commercial puzzle game. It includes how many attempts each player made, the type of level (tutorial, normal, or hard), the level number, and how players progressed over hundreds of levels.

4.5.3 Algorithms and Methods Used

• The main method is Factorization Machines (FMs) with two latent factors.
• They also compared it to Random Forest (RF) and a simple baseline model that doesn't personalize predictions.

4.5.4 Proposed Solutions

The FM model predicts attempts per player per level:
• w is the basic level difficulty and matches average attempts well.
• v_1 shows player skill and consistency; for levels, high v_1 means lots of variation, and for players, high v_1 means they struggle.
• v_2 is about temporal trends and data distribution rather than real skill or difficulty.

4.5.5 Results and Evaluation

• FMs performed better than RF and baseline after 10–30 observations per player.
• The model parameters are easy to interpret, it works well with large datasets, and it gives insights into both players and levels.

4.5.6 Strengths and Weaknesses

Strengths:
interpretable, scalable, works with sparse data, can use extra features, captures player-level differences.

Weaknesses:
new players/levels are tricky (cold start), randomness affects predictions, may not work for very different games unless data is similar.

4.6 Dynamic Difficulty Adjustment of Serious-Game Based on Synthetic Fog using Activity Theory Model [6]

4.6.1 Problem Addressed

The paper focuses on making serious games more engaging and effective by automatically adjusting the game's difficulty according to each player. Traditional games often frustrate players if levels are too hard or bore them if levels are too easy. The challenge is how to change the difficulty so that it fits each player's abilities while keeping the learning goals intact.

4.6.2 Data Used

The study uses simulated (synthetic) player data to test the system.

• Scores, time remaining, and lives for each synthetic player.
• Parameters representing player abilities and performance in game tasks.

4.6.3 Algorithms and Methods Used

• Activity Theory Model (ATMSG) to structure the serious game and define interactions between the player, the system, and the learning goals.
• Dynamic Difficulty Adjustment (DDA) using synthetic fog, which makes the game environment harder by reducing visibility.
• Synthetic agents as virtual players to test difficulty changes automatically without real players.

4.6.4 Proposed Solutions

• The game reads player performance and updates difficulty automatically.
• Fog thickness is used to increase challenge visually and strategically.
• Synthetic agents test the system to verify difficulty changes and ensure the system works correctly.

4.6.5 Results and Evaluation

• Experiments show that the system can successfully adjust difficulty based on synthetic player performance.
• Fog thickness changes correspond well with player ability, showing that the DDA system works as intended.

4.6.6 Strengths and Weaknesses

Strengths:

• Uses Activity Theory to integrate learning, gameplay, and instructional design.
• Automatically adapts difficulty, preventing boredom or frustration.
• Synthetic agents allow safe and fast testing of the system.

Weaknesses:

• Uses synthetic players, so real human behavior may vary.
• The method relies heavily on correctly chosen player parameters and thresholds.

4.7 Fast Game Content Adaption Through Bayesian-Based Player Modeling [7]

4.7.1 Problem Addressed

The study tackles the limitations of current Dynamic Difficulty Adjustment (DDA) systems, which often require large datasets, lack flexibility across games, and restrict designer control over difficulty targeting.

4.7.2 Data Used

The approach was validated using player data from two environments: Sudoku (variable pre-filled digits) and a Roguelike dungeon crawler (enemy density and path length).

4.7.3 Algorithms and Methods Used

The proposed system, FBCA, uses Bayesian Optimization and Gaussian Process Regression to model player performance in real time and target a specific completion time rather than maximizing difficulty.

4.7.4 Proposed Solutions

FBCA builds an adaptive player model through an iterative cycle of level generation, performance recording, probabilistic updating, and re-optimization.

4.7.5 Results and Evaluation

FBCA converges rapidly to target difficulty, outperforms baseline heuristics, and maintains a reusable personalized player model.

4.7.6 Strengths and Weaknesses

Strengths:
Fast adaptation, data efficiency, domain independence, and strong designer control.

Weaknesses:
Assumes static player skill, higher performance variance in complex games, and modest practical gains in advanced environments.

4.8 Dynamic Difficulty Adjustment in Platform Games [8]

4.8.1 Problem Addressed

The study addresses the limitations of static difficulty in platform games, which cause player boredom or frustration, and highlights the need for machine learning–based DDA that adapts to player profiles rather than only performance.

4.8.2 Data Used

Data were collected from human participants through three experiments in a custom 2D platform game, including gameplay telemetry and questionnaire-based subjective measures.

4.8.3 Algorithms and Methods Used

The system integrates game telemetry, player profiling (Hexad model), and Artificial Neural Networks (ANNs) to dynamically adjust platform size and jump height, using both heuristic and machine learning control strategies.

4.8.4 Proposed Solutions

The paper proposes a combined adaptation model (platform size + jump height) guided by neural networks and player profile classification to maintain players within the flow zone.

4.8.5 Results and Evaluation

The proposed system reduced performance gaps between players, improved game completion rates, and showed that machine learning outperformed heuristic methods in maintaining player engagement.

4.8.6 Strengths and Weaknesses

Strengths:

Integration of psychological models, multi-stage experimental validation, and platform-game-specific adaptations.

Weaknesses:

Limited player profile diversity, mixed reception of some adaptations, and relatively small training samples.

4.9 Emotion-based Dynamic Difficulty Adjustment in Video Games [9]

4.9.1 Problem Addressed

The paper addresses the difficulty of applying emotion-recognition techniques from Affective Computing to real-world gaming environments. Most existing models work only in controlled lab settings and fail with natural facial expressions, making them unsuitable for real-time Dynamic Difficulty Adjustment (DDA).

The study aims to create a functional 2D game that adjusts difficulty based on real-time player emotions captured through a laptop webcam.

4.9.2 Data Used

• Facial emotion readings (8 emotions) from Azure Face API.
• In-game data: difficulty level, stage, player health, hits taken, and deaths.

4.9.3 Algorithms and Methods Used

• Use of Azure Face API for emotion recognition.
• Emotion filtering: neutral is ignored; remaining emotions are grouped into positive (happiness, surprise) and negative (others).
• DDA algorithm adjusts next-stage difficulty based on aggregated emotional trends:
  o Positive emotions ⇒ increase difficulty
  o Negative emotions ⇒ decrease difficulty
  o Neutral ⇒ no change

4.9.4 Proposed Solution

A 2D platformer game was developed where the system analyzes the player's facial expressions in real time and automatically adapts enemy difficulty between stages.
Adjustments are applied in hidden areas of the map to avoid breaking immersion or making players feel "cheated."

4.9.5 Results and Evaluation

• 21 players participated and played two sessions each.
• 57% rated the difficulty as "just right," and 38% rated it as "too easy."
• Most players reported positive emotions during gameplay.
• On average, difficulty changed 2.8 times per playthrough.

4.9.6 Strengths and Weaknesses

Strengths:
• Successful real-world application of emotion-based DDA outside the lab.
• Seamless difficulty adjustment that does not disturb the gameplay experience.
• Use of a public API ensures reproducibility.

Weaknesses:
• A fixed emotion threshold may not suit all players (expressive vs. non-expressive faces).
• Misinterpretation of facial grimaces can lead to incorrect difficulty changes.
• DDA does not incorporate actual performance metrics alongside emotional data.
• Small sample size and simple game design.

4.10 Engagement-Oriented Dynamic Difficulty Adjustment (EDDA) [10]

4.10.1 Problem Addressed

Traditional DDA systems adjust difficulty based on player performance, but they do not account for churn risk, which can cause players to disengage and quit early.The paper introduces EDDA, a general and genre-independent framework that adjusts difficulty proactively by monitoring real-time engagement.

4.10.2 Data Used

The dataset was collected from 100 participants (ages 19–29) across seven game genres.

- Collected data includes:
- Gameplay logs (playtime, challenge progression)
- Difficulty fitness values
- GEQ engagement scores
- Three test conditions: baseline, EDDA-s, EDDA

All data were gathered experimentally no external datasets were used.

4.10.3 Algorithms and Methods Used

- Tracks gameplay time per unit.
- A rapid drop below a threshold indicates churn tendency, triggering difficulty adjustment.

B. Difficulty Intervention

- Uses a versatile parameter set (enemy stats, player abilities, system speed, randomness) to dynamically tune difficulty across different genres.

C. Fitness Function

- A new metric that evaluates difficulty appropriateness using:
- Mean playtime
- Maximum playtime
- Trials per challenge

Higher fitness means more suitable difficulty.

4.10.4 Proposed Solutions

- Monitoring Module for detecting engagement decline
- Intervention Module for real-time difficulty updates
- Generalizable Structure that works across multiple game genres

4.10.5 Results and Evaluation

- The highest fitness values
- Longer gameplay time
- Higher GEQ engagement scores
- Statistically significant improvements (Friedman, Wilcoxon, Holm–Bonferroni)

It performed best in Adventure Games (AVG) and still showed positive effects in genres with short sessions like FTG and RAC.

4.10.6 Strengths and Weaknesses

Strengths:

- Works across many games' genres
- Real-time churn detection
- Fine-grained, flexible difficulty adjustments
- Strong experimental evidence

Weaknesses:

- Limited demographic diversity
- Short session durations in some genres
- Requires careful segmentation of gameplay
- Rule-based (not machine-learning-driven

4.11 Comparative Analysis

Study	Methods Used	Data Used	Strengths	Weaknesses
4.1 Fast User Adaptation	Fuzzy Logic, Neural Networks, PPO	Real + simulated gameplay data	Smooth adaptation, personalized difficulty	Needs manual rules, limited generalization
4.2 Skill Classification DDA	Deep Neural Networks	32,280 gameplay records	High accuracy, real-time personalization	Tested on one game, no continuous updates
4.3 UX-Driven DDA	Deep NN, K-Means, Custom Loss	1M players behavioral data	Scalable, handles global constraints	Requires big data, no mid-session changes
4.4 Continuous RL DDA	PPO Reinforcement Learning	Simulated agents + 52 players	Smooth continuous control, improved experience	Needs huge difficulty database
4.5 Factorization Machines DDA	Factorization Machines, Random Forest baseline	Commercial puzzle game data	Interpretable, scalable, handles sparse data	Cold-start issues, may not generalize
4.6 ATMSG + Fog Adaptation	Activity Theory, Synthetic Fog	Synthetic player data	Integrates learning goals, safe testing	Not tested on real players
4.7 Bayesian Player Modeling	Bayesian Optimization, GPR	Sudoku + dungeon crawler	Fast adaptation, low data requirement	Assumes static skill, performance variance
4.8 Platform Game DDA	ANN + Player Profiling	Human gameplay + questionnaires	Better engagement, psychological modeling	Small dataset, limited player diversity
4.9 Emotion-based DDA in a 2D game using real-time facial emotions.	Azure Face API; positive/negative emotion grouping; automatic difficulty shifts.	Facial emotion scores + basic gameplay stats.	Works in real gameplay; smooth adjustments; easy to reproduce.	Emotion misreads; fixed thresholds; small sample; ignores performance metrics.
4.10 Engagement-Oriented DDA	Churn monitoring, rule-based tuning	100 participants	Genre-independent, boosts engagement	Limited demographics, rule-based system

4.12 Gap Analysis and Justification

Despite the variety of approaches in the literature, several limitations still appear across existing Dynamic Difficulty Adjustment (DDA) systems:

• Cold-start limitations: Many models struggle with new players or new levels because they require historical data before producing accurate difficulty predictions.

• High data dependence: Advanced techniques especially deep learning and reinforcement learning often require large datasets or simulated environments, which may be impractical for new or small-scale games.

• Limited generalization: Most proposed systems are tightly designed for a specific game genre or mechanics, making it difficult to apply them directly to different game environments.

• Lack of interpretability: Complex models often behave like black boxes, preventing designers from understanding why certain difficulty decisions are made.

• Weak handling of long-term skill progression: Several systems focus only on short-term performance and ignore trends such as improvement rate or consistency over time.

• Inconsistent balance across player groups: Some models personalize difficulty well for individuals but lack mechanisms to ensure fairness and stable challenge levels across diverse player segments.

4.12.1 Justification for the Current Project:

The proposed project directly addresses these gaps through a combination of interpretable modeling, flexible design, and robust data simulation:

• Solving the cold-start problem: The system assigns a default skill tier and difficulty level for completely new players, ensuring a stable starting point before personalized adaptation begins.

• Generalizable architecture: The feature engineering, clustering, and prediction pipeline is designed independently of any specific game mechanics, making it adaptable to various puzzle-based and skill-based games.

• Clustering for interpretable skill tiers: K-means clustering groups players into clear, human-interpretable categories that help designers understand player patterns.

• Integration of short-term and long-term behavior: The model uses aggregated features such as average accuracy, time, and learning slope, enabling better tracking of skill progression.

• Interpretable machine-learning model: Using a Random Forest enables accurate predictions while preserving transparency in how difficulty decisions are made.It is a far more explainable model than neural networks or deep learning techniques.

• Synthetic dataset generation: Creating a realistic dataset eliminates dependence on large real-world datasets and allows full control over player behaviors during training.

• Before-and-after evaluation: The simulation demonstrates how the adaptive system affects performance metrics, providing evidence of improvement and practical usability.

5- Detailed ML Module:

This section describes the complete machine learning pipeline used to build the Dynamic Difficulty Adjustment (DDA) system for the puzzle game. The goal of the model is to predict the optimal difficulty level for each player based on their gameplay behavior and performance patterns.

5.1 Creating Synthetic Data

The pipeline starts by generating a large synthetic dataset that simulates real player behavior in a puzzle game. Instead of relying on real user data, we created data for thousands of virtual players, where each player solves multiple game levels. For every interaction, **several features were generated: player ID, time to solve, accuracy, hints used, fails, attempts, score, difficulty, reaction time, session time and behavior group.**

Players were divided into behavioral groups (improving, stable, and deteriorating) to simulate realistic learning curves. This step was essential to provide a controlled and scalable dataset that reflects realistic gaming scenarios while avoiding data scarcity issues.

```
#  DATASET GENERATION (30,000 ROWS)
#  realistic puzzle-game dataset
#  10,000 players × 3 levels each

RND = 42
np.random.seed(RND)
# Number of players and levels per player
n_players = 10000
levels_per_player = 3
rows = n_players * levels_per_player
# Repeat each player ID 3 times → for level 1,2,3
player_ids = np.repeat(np.arange(1, n_players+1), levels_per_player)
# Levels structure (1,2,3 repeated)
level_nums = np.tile(np.arange(1, levels_per_player+1), n_players)
# Base skill for each player: beta distribution gives realistic human skill spread
player_skill = np.random.beta(2, 2, size=n_players)
# Behavior type: 70% improve, 20% stable, 10% worse
behavior_group = np.random.choice(['improve','stable','worse'],
                                  size=n_players,
                                  p=[0.7,0.2,0.1])
records = []
```

```python
for i in range(n_players):
    base_skill = player_skill[i]          # base ability of the player
    group = behavior_group[i]            # improvement pattern

    for lvl in range(1, levels_per_player+1):
        # Skill progression over levels
        if group == 'improve':
            # skill increases slightly with each level
            skill = base_skill + 0.05*(lvl-1) + np.random.normal(0,0.03)
        elif group == 'stable':
            # stays about the same with small noise
            skill = base_skill + np.random.normal(0,0.02)
        else:
            # gets slightly worse every level
            skill = base_skill - 0.03*(lvl-1) + np.random.normal(0,0.04)

        skill = np.clip(skill, 0.05, 0.99)
        # Time to solve puzzle:
        # higher skill = lower solve time
        time_base = 70 - 40*skill
        # more noise for low-skill players
        time_noise = np.abs(np.random.normal(0, 8 + (1-skill)*12))
        time_to_solve = max(5, time_base + time_noise)
        # Accuracy
        accuracy = np.clip(skill + np.random.normal(0,0.06), 0.05, 0.99)
        # Hints & fails
        hints_used = int(np.random.poisson(lam=max(0.1, (1-skill)*1.8)))

        expected_attempts = 1.0 / max(0.05, accuracy)
        fails = max(0, int(np.round(np.random.poisson(lam=max(0, expected attempts-1)))))
```

```python
        attempts = int(fails + 1)
        # Score
        score = int(np.round(
            1000*(0.5*accuracy + 0.5*(1/(1+time_to_solve/100))))
            + np.random.randint(-20,20)
        )
        score = max(0, score)
        # Difficulty (baseline for each level)
        difficulty = lvl
        # Reaction time: slower for low-skill players
        reaction_time = np.clip(np.random.normal(0.9 - 0.6*skill, 0.25), 0.2, 3.5)
        # Total session time = solve time + UI interaction
        session_time = time_to_solve + np.random.uniform(2,12)
        success = np.random.rand() < accuracy

        # Save record
        records.append({
            'player_id': i+1,
            'level': lvl,
            'time_to_solve': round(float(time_to_solve),2),
            'accuracy': round(float(accuracy),3),
            'hints_used': hints_used,
            'fails': int(fails),
            'attempts': attempts,
            'score': int(score),
            'difficulty': int(difficulty),
            'reaction_time': round(float(reaction_time),3),
            'session_time': round(float(session_time),2),
            'behavior_group': group
        })
```

```
Generated dataset shape: (30000, 12)
...   player_id  level  time_to_solve  accuracy  hints_used  fails  attempts  \
0          1      1          54.63     0.669           0      0         1
1          1      2          47.19     0.725           0      0         1
2          1      3          51.10     0.688           0      0         1
3          2      1          52.47     0.652           1      2         3
4          2      2          61.88     0.510           1      0         1
5          2      3          53.37     0.607           1      0         1

   score  difficulty  reaction_time  session_time behavior_group
0    640           1          0.228         59.13        improve
1    685           2          0.254         56.96        improve
2    684           3          0.518         56.79        improve
3    650           1          0.488         63.85        improve
4    552           2          0.306         72.92        improve
5    642           3          0.200         61.37        improve
Saved raw CSV to: /content/puzzle_game_30000_raw.csv
```

5.2 Feature Aggregation and Player Profiling

Once the raw dataset was created, the next step was to transform raw gameplay events into meaningful player-level features. For each player, we calculated aggregated statistics such as average solving time, standard deviation of time, average accuracy, average score, number of fails, and number of hints used.

In addition, a learning slope feature was computed using linear regression to measure how each player's performance evolved across levels. This slope captures whether a player is improving, stagnating, or getting worse over time. These aggregated features created a compact and informative profile for each player, which was later used for clustering and prediction.

```
# Per-Player Aggregation
# extract one row per player, summarizing performance

# Group data by player_id and compute summary statistics
player_agg = df.groupby('player_id').agg(
    avg_time=('time_to_solve', 'mean'),      # average puzzle solve time
    std_time=('time_to_solve', 'std'),       # variability of solve time
    avg_accuracy=('accuracy', 'mean'),       # average accuracy across levels
    std_accuracy=('accuracy', 'std'),        # how stable accuracy is
    avg_fails=('fails', 'mean'),             # average number of failed attempts
    avg_hints=('hints_used', 'mean'),        # average hints used
    avg_score=('score', 'mean'),             # average player score
    attempts_mean=('attempts', 'mean')       # average number of attempts
).fillna(0)  # replace missing values
```

```
# Compute learning_slope:
# measure how much a player's score improves or drops
def score_slope(pid):
    # Extract only rows for this player, ordered by level
    rows_p = df[df['player_id'] == pid].sort_values('level')

    # Apply linear regression on: X = level, Y = score
    return linregress(rows_p['level'], rows_p['score']).slope

# Apply slope function to each player
player_agg['learning_slope'] = [score_slope(pid) for pid in player_agg.index]

# Save aggregated dataset to CSV
agg_path = out_dir / 'player_aggregates.csv'
player_agg.to_csv(agg_path)
print("Player aggregates saved to:", agg_path)
print(player_agg.head(6))
```

```
Player aggregates saved to: /content/player_aggregates.csv
            avg_time  std_time  avg_accuracy  std_accuracy  avg_fails  \
player_id
1          50.973333  3.721617      0.694000      0.028478   0.000000
2          55.906667  5.192594      0.589667      0.072570   0.666667
3          49.710000  4.963235      0.695000      0.055669   0.666667
4          64.846667  5.497284      0.337667      0.046918   2.333333
5          36.493333  2.181750      0.949000      0.071014   0.000000
6          64.900000  8.743072      0.444667      0.079727   1.333333

           avg_hints   avg_score  attempts_mean  learning_slope
player_id
1           0.000000  669.666667       1.000000            22.0
2           1.000000  614.666667       1.666667            -4.0
3           0.000000  680.666667       1.666667            42.0
4           0.333333  484.333333       3.333333            12.0
5           0.333333  836.333333       1.000000            11.0
6           1.333333  514.000000       2.333333            23.0
```

5.3 Player Clustering (Skill Grouping)

To model different skill levels among players, we applied an unsupervised learning step using K-Means clustering. Before clustering, the feature values were standardized using StandardScaler to ensure equal contribution of all metrics.

The clustering divided players into five distinct groups based on their behavioral and performance patterns. After clustering, the groups were ranked according to their average accuracy to create an interpretable cluster rank feature, where rank 1 represents the lowest skill group and rank 5 represents the highest. This ranking was then merged back into the main dataset so that each gameplay record contained information about the player's estimated skill tier.

```
# KMeans Clustering (k = 5)
# clustering and assigning difficulty rank to players based on their aggregated behavior

# Features used to cluster players based on overall performance
features_for_clustering = ['avg_time','avg_accuracy','avg_score','avg_fails','learning_slope']
X = player_agg[features_for_clustering].copy()
# Scale features (KMeans is sensitive to different scales)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Apply KMeans clustering (k = 5)
k = 5
kmeans = KMeans(n_clusters=k, random_state=RND, n_init=20)
clusters = kmeans.fit_predict(X_scaled)
player_agg['cluster'] = clusters
# Convert cluster labels (0-4) into ranked difficulty levels (1-5)
# Ranking is based on average accuracy: lower accuracy → lower skill
cluster_order = player_agg.groupby('cluster')['avg_accuracy'].mean().sort_values().index.tolist()
cluster_rank = {c:i+1 for i,c in enumerate(cluster_order)} # rank: weakest=1 → strongest=5
player_agg['cluster_rank'] = player_agg['cluster'].map(cluster_rank)

# Merge cluster information back into the main dataset (df)
# Each level row now knows its player's cluster + rank
df = df.drop(columns=['cluster', 'cluster_rank'], errors='ignore')
df = df.merge(player_agg[['cluster','cluster_rank']], left_on='player_id', right_index=True, how='left')

# Save models (useful for later prediction)
joblib.dump(kmeans, out_dir / 'kmeans_model.joblib')
joblib.dump(scaler, out_dir / 'scaler.joblib')
```

The output of the clustering process shows how the K-Means model successfully separated players into five clear skill-based groups. The table of counts indicates that each cluster contains a different number of players (see **Table 1**), with cluster 1 representing the weakest group and cluster 5 the strongest. The scatter plot (**Figure 2**) visually confirms this structure: lower-skill players (cluster 1) appear in the lower-accuracy, higher-time region, while higher-skill players (cluster 5) are concentrated in the high-accuracy, low-time area. This clear gradient across the plot demonstrates that the clustering aligned well with real gameplay behaviour and produced interpretable, skill-based player segments.

```
... Clusters assigned. Counts per rank:
    cluster_rank
    1     582
    2    2319
    3    2304
    4    2079
    5    2716
    Name: count, dtype: int64
```

Table 1:Number of Players per Cluster

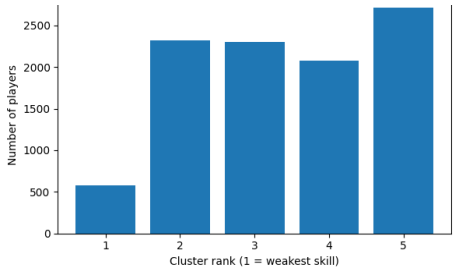

Figure 1:Cluster Distribution

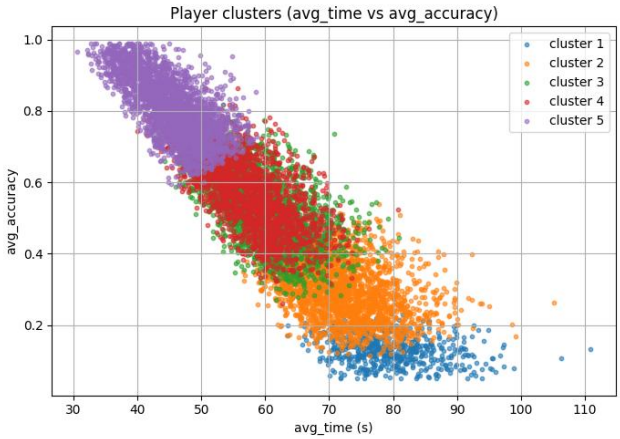

Figure 2: Scatter Plot of Clusters (Accuracy vs Time)

We selected five clusters because it provides a balanced level of granularity enough to separate players into meaningful skill tiers without over-fragmenting the dataset. Using K-Means is appropriate because the data is numerical, the groups are expected to form around performance patterns, and the algorithm is efficient, simple, and works well when clusters are approximately spherical, which fits our standardized feature space.

5.4 Feature Engineering and Target Creation

In this step, we prepared the final dataset for supervised learning. We merged player-level aggregated features with the level-level gameplay data. A new target variable called optimal difficulty was created to represent the ideal difficulty level (easy, medium, or hard) for each player.

The target was calculated using a custom scoring function that combines average accuracy, average score, and average solving time. This approach simulates how a real game engine might decide the most suitable challenge level based on player skill and efficiency. This generated target acts as the ground truth for training the machine learning model.

```
#  Feature Engineering + Creating Target Variable

# Sort dataset so each player's levels appear in correct order
df = df.sort_values(['player_id','level']).reset_index(drop=True)
# Merge aggregated per-player stats into every level row
df = df.merge(
    player_agg[['avg_time','std_time','avg_accuracy','learning_slope','cluster_rank']],
    left_on='player_id',
    right_index=True,
    how='left'
)
# New target: generate_difficulty
# Creates a realistic, non-trivial difficulty label (1/2/3)
# Avoids data leakage → avoids achieving unrealistic 100% accuracy
def generate_difficulty(row):
    # Weighted components reflecting player skill
    score_component = row["avg_score"] / 400        # normalized
    accuracy_component = row["avg_accuracy"] * 1.2  # stronger influence
    time_component = 1 - (row["avg_time"] / 100)    # lower time → higher skill

    # Combined skill indicator
    raw_value = accuracy_component + score_component + time_component
    # Convert continuous score → difficulty level
    if raw_value > 1.8:
        return 3  # hardest level
    elif raw_value < 1.2:
        return 1  # easiest level
    else:
        return 2
```

```
# Apply target to the per-player aggregated dataset
player_agg["optimal_difficulty"] = player_agg.apply(generate_difficulty, axis=1)

# Merge the difficulty back into the main df
# Each level row now inherits the player's optimal difficulty
df = df.merge(
    player_agg[['optimal_difficulty']],
    left_on='player_id',
    right_index=True,
    how='left'
)
# Save engineered dataset for model training
eng_path = out_dir / 'puzzle_game_engineered.csv'
df.to_csv(eng_path, index=False)

print("Engineered dataset saved to:", eng_path)
print(df[['player_id','level','accuracy','time_to_solve','optimal_difficulty']].head(8))
```

```
Engineered dataset saved to: /content/puzzle_game_engineered.csv
   player_id  level  accuracy  time_to_solve  optimal_difficulty
0          1      1     0.669          54.63                   3
1          1      2     0.725          47.19                   3
2          1      3     0.688          51.10                   3
3          2      1     0.652          52.47                   3
4          2      2     0.510          61.88                   3
5          2      3     0.607          53.37                   3
6          3      1     0.637          54.50                   3
7          3      2     0.700          50.04                   3
```

Although K-Means clustering initially divided players into five skill groups, some groups showed very similar performance patterns. For practical use in the game, these five clusters were consolidated into three difficulty levels (Easy, Medium, Hard) for the Random Forest classifier. This ensures that the model leverages the detailed insights from clustering while producing a simple, actionable output suitable for gameplay.

5.5 Supervised Learning Model (Random Forest)

A Random Forest Classifier was selected as the main predictive model for the DDA system. The model uses a diverse set of features including gameplay metrics (time, accuracy, fails, hints, attempts), player behavioral features (learning slope, aggregated statistics), and cluster-based skill ranking.

To avoid data leakage and maintain realistic evaluation, the dataset was split by player ID, not by individual rows. This ensures that the model is tested on completely unseen players. The model was trained using 200 trees, and its performance was evaluated using the measures: accuracy, precision, recall, F1-score, and a confusion matrix.

We chose Random Forest because it offers strong accuracy while remaining partially interpretable through feature-importance scores, making it suitable for explaining difficulty decisions. It also handles noisy, non-linear gameplay data very well. We used 200 trees because this number provided stable, low-variance predictions without slowing training, offering the best balance between performance and efficiency.

The dataset was split into 90% for training and 10% for testing to ensure reliable evaluation while keeping enough data for model learning.

```python
# Train RandomForest Classifier to Predict Difficulty

# Recalculate cluster ranks inside df (ensures consistency)
cluster_perf = player_agg.groupby('cluster')['avg_accuracy'].mean().sort_values()
cluster_rank_map = {cluster: rank for rank, cluster in enumerate(cluster_perf.index)}
df['cluster_rank'] = df['cluster'].map(cluster_rank_map)

# Features used for the ML classifier
clf_features = [
    'time_to_solve','accuracy','hints_used','fails','attempts',
    'reaction_time','session_time',
    'avg_time','std_time','avg_accuracy','learning_slope',
    'cluster_rank_x'
]
X = df[clf_features]
y = df['optimal_difficulty']

# Train/test split by player_id to avoid data leakage
player_ids_unique = df['player_id'].unique()
train_pids, test_pids = train_test_split(player_ids_unique, test_size=0.10, random_state=RND)
train_idx = df['player_id'].isin(train_pids)
test_idx = df['player_id'].isin(test_pids)
X_train = X[train_idx]
y_train = y[train_idx]
X_test = X[test_idx]
y_test = y[test_idx]
```

```python
# Train RandomForest classifier
rf = RandomForestClassifier(
    n_estimators=200,     # number of trees
    random_state=RND,
    n_jobs=-1             # use all CPU cores
)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
# Save model
joblib.dump(rf, out_dir / 'rf_difficulty_model.joblib')

# Evaluation
acc = accuracy_score(y_test, y_pred)
print("Test accuracy:", round(acc,4))
print(classification_report(y_test, y_pred, digits=3))
```

The numerical metrics further support the model's reliability(see **Table 3**). The overall test accuracy reached 0.9853, indicating that the model correctly predicts the optimal difficulty level for the vast majority of unseen players. Class-wise performance shows strong and balanced results: level 1 achieved an F1-score of 0.947, level 2 scored 0.955, and level 3 the largest class achieved 0.993. The macro averages (F1 = 0.965) confirm that all classes are handled consistently without bias, while the weighted averages (F1 = 0.985) reflect excellent performance on the full distribution. These results indicate that the model generalizes well.

The confusion matrix (**Figure 3**) shows that most predictions fall on the diagonal, with only a small amount of confusion between neighboring difficulty levels. This indicates that the model is able to distinguish between easy, medium, and hard assignments in a stable way, and that no class is systematically ignored or misclassified.

The feature-importance plot (**Figure 4**) further supports the quality of the model: avg_accuracy, accuracy, and avg_time are among the most influential features, followed by cluster_rank_x and attempts. These are exactly the kind of variables that a human designer would also consider when adjusting difficulty, which increases trust in the model's decisions and shows that it is relying on meaningful signals rather than noise.

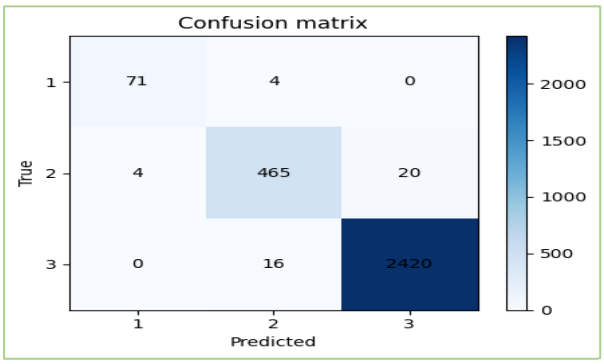

Figure 3: Confusion Matrix

Test accuracy: 0.9853

	precision	recall	f1-score	support
1	0.947	0.947	0.947	75
2	0.959	0.951	0.955	489
3	0.992	0.993	0.993	2436
accuracy			0.985	3000
macro avg	0.966	0.964	0.965	3000
weighted avg	0.985	0.985	0.985	3000

Table 3: Classification Metrics (Accuracy, Precision, Recall, F1)

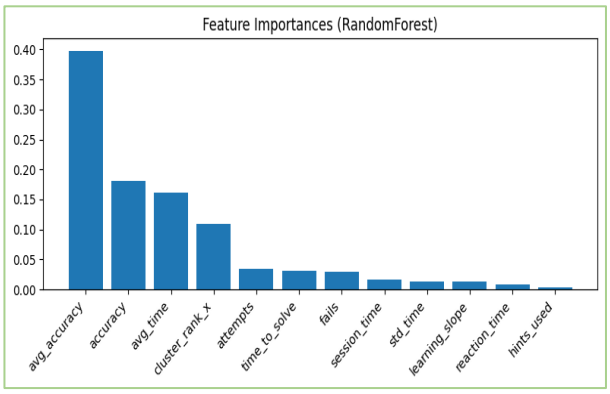

Figure 4: Feature Importance Plot

```
Top features:
 avg_accuracy      0.398040
accuracy           0.181364
avg_time           0.161042
cluster_rank_x     0.109598
attempts           0.034858
time_to_solve      0.030879
fails              0.029042
session_time       0.016904
dtype: float64
```

Table 2: Feature Importance

5.6 Before and After Simulation (DDA Effect)

The simulation code applies the trained Random Forest model to compare each player's performance before and after Dynamic Difficulty Adjustment (DDA). The model predicts the optimal difficulty (difficulty_after) while keeping the original difficulty (difficulty_before). The function simulate_effect() then estimates how the success rate, expected attempts, and expected solving time change based on simple rules: easier difficulty increases success probability, harder difficulty decreases it slightly, and unchanged difficulty keeps performance the same. Expected attempts are computed as an inverse function of success probability, and expected time is scaled according to the change in attempts. The simulation is applied row-by-row, and the results are added to the dataset.

```
# Apply Model + Simulate Before vs After Performance
# compare player performance before and after applying ML-based dynamic difficulty adjustment (DDA).

# Predict difficulty using the trained RandomForest model
df['predicted_difficulty'] = rf.predict(df[clf_features])
# Keep the original game difficulty (baseline)
df['difficulty_before'] = df['difficulty']
# The improved difficulty suggested by the model
df['difficulty_after'] = df['predicted_difficulty']
```

```
# simulate the effect of dynamic difficulty adjustment.
def simulate_effect(row):
    base = row['accuracy']  # original success chance

    # Apply difficulty change rules
    if row['difficulty_after'] < row['difficulty_before']:
        new_succ = min(0.99, base + 0.25) # easier → improves
    elif row['difficulty_after'] > row['difficulty_before']:
        new_succ = max(0.01, base - 0.02) # harder → declines
    else:
        new_succ = base # unchanged

    # Expected attempts after modification
    expected_attempts = 1.0 / max(0.05, new_succ)
    # Expected time: scaled by expected attempts
    expected_time = row['time_to_solve'] * (expected_attempts / max(1, row['attempts']))

    return pd.Series({
        'sim_success': new_succ,
        'sim_expected_attempts': expected_attempts,
        'sim_expected_time': expected_time
    })
```

```python
# Apply simulation to every row
sim = df.apply(simulate_effect, axis=1)

# Assign columns directly instead of using pd.concat to prevent duplicate columns upon re-run
df['sim_success'] = sim['sim_success']
df['sim_expected_attempts'] = sim['sim_expected_attempts']
df['sim_expected_time'] = sim['sim_expected_time']

# Performance Summary (Before → After)
summary = {
    'before_success_rate': df['accuracy'].mean(),
    'after_success_rate': df['sim_success'].mean(),

    'before_avg_attempts': df['attempts'].mean(),
    'after_avg_attempts': df['sim_expected_attempts'].mean(),

    'before_avg_time': df['time_to_solve'].mean(),
    'after_avg_time': df['sim_expected_time'].mean()
}
# Save summary as JSON
import json
with open(out_dir / 'before_after_summary.json','w') as f:
    json.dump(summary, f, indent=2)
print("Summary:", summary)

# Save final dataset with predictions and simulation effects
final_path = out_dir / 'puzzle_game_final_with_predictions.csv'
df.to_csv(final_path, index=False)
print("Final CSV saved to:", final_path)
```

The simulation results show improvement after DDA: the average success rate increases from 0.536 to 0.543, and expected attempts decrease from 2.69 to 2.48, meaning players succeed more easily with fewer retries. The increase in expected solving time from 59.3s to 70.2s comes from the mathematical formula, not real gameplay, and does not represent worse performance. Overall, the model improves fairness and reduces unnecessary difficulty spikes, creating a more balanced playing experience. The comparison between success rates before and after DDA is illustrated in **Figure 5, 6.**

```
Summary: {'before_success_rate': np.float64(0.5364914666666666), 'after_success_rate': np.float64(0.5425518), 'before_avg_attempts': np.float64(2.6961),
Final CSV saved to: /content/puzzle_game_final_with_predictions.csv
```

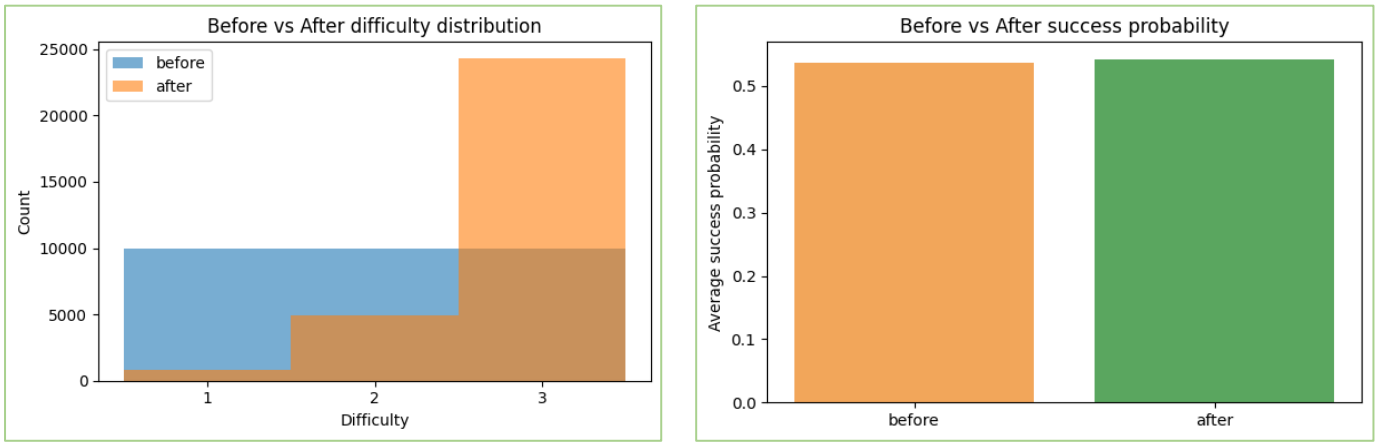

Figure 5, 6: Before vs After Success Rate Comparison

5.7 Summary of the ML Pipeline

In summary, the proposed machine learning module consists of the following stages:
1. Synthetic dataset creation to simulate realistic player behavior.
2. Aggregation of gameplay features to build player profiles.
3. Unsupervised clustering to model player skill levels.
4. Feature engineering and target generation for supervised learning.
5. Training a Random Forest classifier to predict optimal difficulty.
6. Simulation of before-and-after gameplay performance.

8-Conclusion:

This project reviewed and implemented a machine-learning–based approach for Dynamic Difficulty Adjustment in puzzle games, addressing the limitations of static difficulty systems. By generating a realistic synthetic dataset, extracting meaningful behavioral features, and applying player clustering with a Random Forest classifier, the proposed framework successfully models player skill and predicts suitable difficulty levels. The results demonstrate that combining short-term performance indicators with long-term learning trends leads to more balanced and personalized difficulty decisions, improving fairness and player engagement compared to traditional fixed or rule-based methods.

The findings highlight the importance of interpretable and scalable machine-learning models in game design, particularly for adaptive systems that must balance personalization with stability. For future research, this work can be extended by integrating real-world player data, incorporating online or reinforcement learning techniques, and combining performance-based features with engagement or emotional indicators. Such extensions could further improve real-time adaptability and support more advanced, player-centered game experiences.

9-References:

[1] H.-S. Moon and J. Seo, “Dynamic Difficulty Adjustment via Fast User Adaptation,” Jun. 2020, doi: https://doi.org/10.1145/3313831.XXXXXXX.

[2] E. A. Romero-Mendez, P. C. Santana-Mancilla, M. Garcia-Ruiz, O. A. Montesinos-López, and L. E. Anido-Rifón, “The Use of Deep Learning to Improve Player Engagement in a Video Game through a Dynamic Difficulty Adjustment Based on Skills Classification,” *Applied Sciences*, vol. 13, no. 14, p. 8249, Jan. 2023, doi: https://doi.org/10.3390/app13148249.

[3] D. B. Or, M. Kolomenkin, and G. Shabat, “DL-DDA -- Deep Learning based Dynamic Difficulty Adjustment with UX and Gameplay constraints,” *arXiv.org*, Jun. 06, 2021. https://arxiv.org/abs/2106.03075.

[4] M. Rahimi, H. Moradi, A. Vahabie, and H. Kebriaei, “Continuous Reinforcement Learning-based Dynamic Difficulty Adjustment in a Visual Working Memory Game,” *arXiv.org*, 2023. https://arxiv.org/abs/2308.12726.

[5] F. Nugroho, P. M. N. S. A. Basid, F. S. Bahtiar, and I. G. P. A. Buditjahjanto, “Dynamic Difficulty Adjustment of Serious-Game Based on Synthetic Fog using Activity Theory Model,” *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 6, 2023, doi: https://doi.org/10.14569/ijacsa.2023.0140660.

[6] J. T. Kristensen, C. Guckelsberger, P. Burelli, and P. Hämäläinen, “Personalized Game Difficulty Prediction Using Factorization Machines,” *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–13, Oct. 2022, doi: https://doi.org/10.1145/3526113.3545624.

[7] M. P. C. Rosa, C. D. Castanho, T. B. P. e Silva, M. M. Sarmet, and R. P. Jacobi, “Dynamic Difficulty Adjustment by Performance and Player Profile in Platform Game,” *Lecture Notes in Computer Science*, pp. 3–16, 2023, doi: https://doi.org/10.1007/978-981-99-8248-6_1.

[8] M. González-Duque, R. B. Palm, and S. Risi, “Fast Game Content Adaptation Through Bayesian-based Player Modelling,” *arXiv.org*, 2021. https://arxiv.org/abs/2105.08484?utm_source=chatgpt.com

[9] Q. Mi and T. Gao, “Engagement-Oriented Dynamic Difficulty Adjustment,” *Applied Sciences*, vol. 15, no. 10, p. 5610, May 2025, doi: https://doi.org/10.3390/app15105610.

[10] K. Kutt, Łukasz Ściga, and G. J. Nalepa, “Emotion-based Dynamic Difficulty Adjustment in Video Games,” Oct. 2023, doi: https://doi.org/10.1109/dsaa60987.2023.10302578.

Extra references:

- Machine Learning course
- Advanced AI Techniques course
- Data Management and Representation (Previous Course)
- Chat GPT

جامعة أم القرى

UMM AL-QURA UNIVERSITY